



Degree Project in Computer Science and Engineering

First cycle, 15 credits

A comparative study on Shor's quantum algorithm and classical integer factorization algorithms based on runtime and accuracy

NIMA CHERAGHI DEHDEZI

HARALAMPOS TRICOPOULOS

A comparative study on Shor's quantum algorithm and classical integer factorization algorithms based on runtime and accuracy

NIMA CHERAGHI DEHDEZI
HARALAMPOS TRICOPOULOS

Degree Project in Computer Science and Engineering, First Cycle
15 credits

Date: June 30, 2025

Supervisor: Bo Peng

Examiner: Pawel Herman

School of Electrical Engineering and Computer Science

Swedish title: En jämförande studie av Shors algoritm och
klassiska heltalsfaktoriseringsalgoritmer baserat på exekveringstid
och noggrannhet

Abstract

Quantum computing has sparked much interest and discussion in recent years due to its potential in speeding up computations and algorithms that can revolutionize multiple technical fields. In this study, we have used a quantum integer factorization algorithm and compared it with classical algorithms in terms of runtime and accuracy. In particular, the comparison involved Shor's algorithm against Trial division and Pollard's Rho algorithm for factoring the integer $N = 15$. IBM's Qiskit library in Python was used to implement and simulate the quantum circuits necessary to run Shor's algorithm. The Shor's implementation was coded specifically for factoring $N = 15$ due to larger integers requiring significantly more resources to simulate. It was found that Shor's algorithm was slower and less accurate than the classical counterparts. This was due to using a simulator instead of real quantum hardware and only being able to run the algorithm on a small integer value such as $N = 15$. However, in the future, if a similar study is conducted on real quantum hardware and on larger integers, it is possible that the results will be vastly different.

Sammanfattning

Kvantberäkningar har väckt mycket intresse och diskussion under de senaste åren på grund av dess möjligheter att effektivisera beräkningar och algoritmer inom flera tekniska områden. I denna studie har vi använt en kvantalgoritm för faktorisering av heltal och jämfört denna med klassiska algoritmer utifrån exekveringstid och noggrannhet. Jämförelsen i fråga involverade Shors algoritm gentemot *Trial division* (eng.) och Pollards Rho-algoritm för att faktorisera heltalet $N = 15$. IBM:s Qiskit bibliotek i Python användes i syfte att implementera och simulera kvantkretsarna som var nödvändiga för att köra Shors algoritm. Implementationen av Shors algoritm kodades specifikt för faktorisering av $N = 15$ på grund av att simuleringen av större heltal kräver betydligt mer resurser. Det visade sig att Shors algoritm var långsammare och mindre precis än de klassiska algoritmerna. Detta orsakades till följd av att en simulator användes istället för faktisk kvanthårdvara och att faktoriseringen tillämpades på ett litet heltalsvärde som $N = 15$. Dock, om en liknande framtida studie genomförs på kvanthårdvara och på större heltal skulle det vara möjligt att resultaten blir betydligt olika.

Acknowledgements

We would like to express our gratitude to our supervisor, Bo Peng, for her valuable support and guidance throughout the process of writing this thesis. We would also like to acknowledge and thank our peers for reviewing our thesis and providing constructive feedback.

Contents

1	Introduction	1
1.1	Aims and Purpose	1
1.2	Scope	2
1.3	Approach	2
2	Theoretical Background	3
2.1	Qubits	3
2.2	Quantum Gates	4
2.3	Phase	6
2.4	Quantum Fourier Transform	6
2.5	Quantum Phase Estimation	7
2.6	Shor’s Algorithm	8
2.7	Classical Algorithms	11
2.7.1	Trial division	11
2.7.2	Pollard’s Rho Algorithm	12
2.8	Related work	13
3	Method	16
3.1	Circuits and algorithms	16
3.2	Experimental setup/Implementation	18
3.3	Evaluation of the metrics	19
4	Results	21
5	Discussion	25
5.1	Societal impact and Industrial relevance	27
5.2	Ethical aspects and Sustainability	27
5.3	Limitations and Future work	28
6	Conclusion	29

A Source Code	34
------------------------	-----------

Chapter 1

Introduction

Computer technology has had many achievements during the past fifty years of advancements, leading to the present day classical computers. However, with increasing computational demands for heavy tasks within cryptography and science, classical computers face certain struggles [1] [2]. Problems requiring heavy computational power such as integer factorization with extremely large numbers remain difficult and unfeasible for classical computers which inherently has resulted in an alternative computing paradigm known as quantum computing gaining interest [3].

Quantum computing utilizes quantum mechanical concepts such as superposition and entanglement in order to efficiently perform computations. This technology offers potential advantages over traditional computers by incorporating quantum circuits which result in drastic speedups that reduce the execution time of algorithms [2]. Quantum algorithms can be applied in real world problems such as integer factorization of large numbers in which currently the classical algorithms and hardware have a difficult time with. Shor's algorithm is a widely recognized quantum integer factorization algorithm developed by Peter Shor, which in theory can factor large numbers in polynomial time. This has sparked an interest in the quantum world due to the algorithm's potential of being able to break encryption methods such as RSA encryption that build on the concept of large prime numbers [2] [4].

1.1 Aims and Purpose

This study aims to evaluate Shor's quantum integer factorization algorithm and compare it to classical factorization algorithms in order to acquire a deeper un-

derstanding of how quantum computing can enhance factorization efficiency. The research question is as follows:

How does Shor's algorithm compare to classical algorithms in integer factorization from the perspective of accuracy and runtime?

1.2 Scope

This study will be focusing on evaluating Shor's algorithm using a quantum simulation environment rather than real quantum hardware. The research will implement a specific quantum circuit for factoring a small integer N due to feasibility. This is because of the complexity and resource demands behind an implementation of Shor's algorithm for larger integers. Specifically, the circuit will be implemented for the purpose of factoring the integer 15 which is commonly used to be able to fully execute and analyze the algorithm. During the experiment, the algorithm's performance and accuracy in the simulation will be analyzed and the results will be compared to classical factorization methods.

1.3 Approach

The experiment will be conducted in a Python environment and will make use of the Qiskit SDK framework in order to implement the quantum circuits and algorithms. The quantum circuits have the option of being executed on real quantum hardware with the use of the IBM Cloud Services. However, in our case, the quantum circuits will be executed locally on our own hardware through Qiskit's quantum simulator. It should be noted that the classical algorithms will also be implemented in Python. All the test runs will be organized in a Jupyter Notebook which divides the code into different code cells. This makes it easier to structure and visualize the results. Finally, the algorithms will be run a set number of times for the purpose of obtaining statistically significant results.

Chapter 2

Theoretical Background

This segment contains information about quantum computing, the theory behind Shor's algorithm and its classical counterparts. Initially, fundamental concepts in quantum computing are introduced, such as qubits, quantum states, gates and phase. Furthermore, project specific concepts are explained, including Quantum Fourier Transform and Quantum Phase Estimation which lay the foundations for Shor's algorithm. Lastly, this segment concludes with details regarding Shor's algorithm and the classical algorithms (Trial division and Pollard Rho's algorithm).

2.1 Qubits

Classical computing works with bits that are binary, meaning that they can only be 0 or 1. Quantum computing instead works with qubits (short for quantum bits) which have both similarities and differences to the traditional bits. The main similarity is that when measuring qubits, they will either be in the 0 state or the 1 state, which are also known as the basis states. Usually, the state of a qubit is written in Dirac notation. Therefore, the 0 and 1 states can be written as $|0\rangle$ and $|1\rangle$, which can be seen in (2.1) [5] [6].

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (2.1)$$

A core difference between qubits and binary bits is that the former can be $|0\rangle$ and $|1\rangle$ simultaneously, and when this occurs the qubit is said to be in a superposition. Qubits can only be in superposition before their measurement occurs. Afterwards, the result is exclusively 0 or 1 [5].

The Dirac notation for a qubit in superposition includes factors that are referred to as amplitudes, as can be seen in (2.2). These amplitudes, written as α and β in (2.2), are complex numbers that indicate the probability of the qubit's state being measured as $|0\rangle$ or $|1\rangle$. The probability of measuring $|0\rangle$ or $|1\rangle$ is $|\alpha|^2$ and $|\beta|^2$ respectively, where $|\alpha|^2 + |\beta|^2 = 1$ [5].

$$|\Psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad (2.2)$$

2.2 Quantum Gates

Another way to visualize the state of qubits is by using a Bloch sphere. This is a unit sphere with a vector inside of it, the position of which represents the state of the qubit. Figure 2.1, illustrates the Bloch spheres for the $|0\rangle$ and $|1\rangle$ states. From this figure it becomes clear that $|0\rangle$ and $|1\rangle$ point to the poles of the sphere [7].

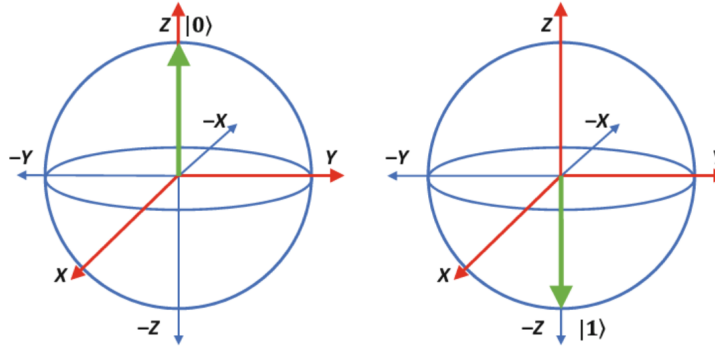


Figure 2.1: Two Bloch spheres showcasing the quantum states for $|0\rangle$ and $|1\rangle$. [7]

There are various gates that can be applied on qubits to change their state. They can be represented as unitary matrices which gives gates two important properties. Firstly, that the sum of probabilities will total 1 and secondly, that every gate has an inverse [5] [8]. Some notable quantum gates are the Pauli-X, Pauli-Y, and Pauli-Z gates. Geometrically speaking, all of these rotate a qubit's state vector π radians around the corresponding axis in the Bloch sphere. The Pauli-X gate, which can be seen in (2.3), is the quantum equivalent to the classical NOT gate since it changes the state $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$ [6].

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (2.3)$$

Another important gate is the Hadamard gate which can be seen in (2.4). The effect of this gate differs depending on which qubit state it is applied on. When it is applied on $|0\rangle$, then the state becomes $|+\rangle$, see (2.5), and if the gate is applied on $|1\rangle$ then the state becomes $|-\rangle$, see (2.6) [5]. This becomes relevant when discussing the phase of qubits.

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (2.4)$$

$$H |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \quad (2.5)$$

$$H |1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \quad (2.6)$$

Up until now qubits and the effect that gates have upon them have been looked at individually. However, some gates need multiple qubits to be implemented. Tensor products are used when working with states of qubit systems (the tensor symbol being $\otimes$) which can be seen in (2.7) [7].

$$|0\rangle \otimes |0\rangle = |00\rangle \quad (2.7)$$

The probability of measuring a particular quantum state in a system of qubits depends on the amplitudes the qubits had prior to the calculation of the tensor product. This can be seen in (2.8) where the tensor product of two qubits is calculated and the amplitudes $(\alpha_1, \beta_1, \alpha_2, \beta_2)$ of the various states get multiplied with each other which results in new amplitudes. It is worth mentioning that similarly to when working with a single qubit, the amplitudes of the qubit system need to satisfy the following relation: $|\alpha_1\alpha_2|^2 + |\alpha_1\beta_2|^2 + |\beta_1\alpha_2|^2 + |\beta_1\beta_2|^2 = 1$ [7].

$$\begin{aligned} &(\alpha_1 |0\rangle + \beta_1 |1\rangle) \otimes (\alpha_2 |0\rangle + \beta_2 |1\rangle) = \\ &\alpha_1\alpha_2 |00\rangle + \alpha_1\beta_2 |01\rangle + \beta_1\alpha_2 |10\rangle + \beta_1\beta_2 |11\rangle \end{aligned} \quad (2.8)$$

One general example of a gate that affects multiple qubits is the controlled-U gate, where U is some unitary matrix. This gate uses one qubit as a control and can have multiple other qubits as targets that will be affected by the gate U only if the control is 1. If the control is 0, the targets will remain unaffected [8].

Another important gate worth mentioning that affects multiple qubits is the SWAP gate, see (2.9). This gate operates on two qubits and makes them swap

their quantum states with each another [7].

$$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.9)$$

2.3 Phase

There are two kinds of phase, global and relative, and they are usually represented with numbers in exponential form. If there is a state $|\Psi\rangle$, then having a global phase factor $e^{i\theta}$ ($\theta \in R$) does not affect calculations. This is because statistically $e^{i\theta}|\Psi\rangle$ and $|\Psi\rangle$ will lead to the same measurements. Therefore, the global phase can be discarded in calculations. However, relative phase is observable. Assuming there are two states, $a|0\rangle + b|1\rangle$ and $c|0\rangle + d|1\rangle$, where a, b, c and d are all amplitudes, then there will exist a difference in relative phase between the amplitudes for $|1\rangle$ if a number $\epsilon \in R$ satisfies $b = e^{i\epsilon}d$. The same idea can be applied to $|0\rangle$. In the case of the $|+\rangle$ and $|-\rangle$ states, when focusing on $|1\rangle$, $b = 1$ and $d = -1$ which means that the relative phase factor must equal -1 [8]. This also means that the relative phase factor is $-1 = e^{i\pi}$. It should be noted that when working with the phase of states, the relative phase can be interpreted as an angle. In the case of the minus sign, it is associated with angle π [20]. When the relative phase for some basis is the sole difference between states, then this leads to them having their own respective measurement statistics. Therefore, such states cannot be viewed as identical. [8].

2.4 Quantum Fourier Transform

The quantum Fourier transform (QFT) is a process which enables the encoding of numbers into the relative phase of states in uniform superposition [9]. As can be seen in the circuit for QFT, depicted in Figure 2.2, all n input qubits will at some point be the target and/or control for a controlled R gate. The R gate affects the phase of the target qubit if applied and the amount depends on the value of k , see (2.10) [8].

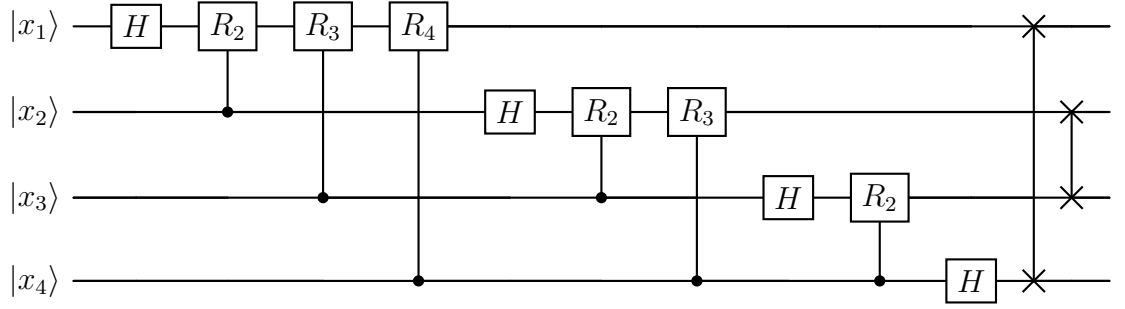


Figure 2.2: A circuit showing the implementation of QFT with 4 qubits.

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{2\pi i}{2^k}} \end{bmatrix} \quad (2.10)$$

It is important to mention that all qubits are put in superposition (due to the Hadamard gates) prior to the application of any R gates. The first qubit interacts with the controlled R gates R_2 to R_n , the second qubit with R_2 to R_{n-1} etc. After all controlled R gates have been applied, swap gates change the positions of the qubits. The final state can be seen in (2.11), where $x_1, x_2, \dots$, and x_n each correspond to the state of the first, second, ..., and n -th input qubit respectively. [8].

$$\begin{aligned} & \frac{1}{\sqrt{2^n}} (|0\rangle + e^{2\pi i(0.x_n)} |1\rangle) \otimes \dots \\ & \dots \otimes (|0\rangle + e^{2\pi i(0.x_2 \dots x_n)} |1\rangle) \otimes \\ & (|0\rangle + e^{2\pi i(0.x_1 x_2 \dots x_n)} |1\rangle) \end{aligned} \quad (2.11)$$

There is also an inverse quantum Fourier transformation IQFT which takes a number in QFT form and changes it to the computational basis [9].

2.5 Quantum Phase Estimation

Quantum phase estimation (QPE) is an algorithm for finding the eigenvalue $e^{2\pi i\theta}$ for an eigenvector $|\psi\rangle$ of a unitary matrix U . This is equivalent to estimating the phase θ . The relation between U and $|\psi\rangle$ can be seen in (2.12) where $0 \leq \theta \leq 1$.

$$U |\psi\rangle = e^{2i\theta} |\psi\rangle \quad (2.12)$$

QPE is initiated by placing all n qubits of the counting register to superposition through Hadamard gates. The application of controlled U gates on $|\psi\rangle$, which is stored at the ancilla register, leads to phase kickback which affects phase of the qubits in the counting register. After all the U gates have been applied, the state becomes (2.13) and the contents of the counting register get turned to the computational basis by using IQFT. Afterwards, measurements are made and then θ can be approximated [7] [10].

$$\begin{aligned} & \frac{1}{\sqrt{2}}((|0\rangle + (e^{2\pi i \theta 2^{n-1}} |1\rangle)) |\psi\rangle \otimes \dots \\ & \dots \otimes (|0\rangle + e^{2\pi i \theta 2^1} |1\rangle) |\psi\rangle \otimes \\ & (|0\rangle + (e^{2\pi i \theta 2^0} |1\rangle) |\psi\rangle) \end{aligned} \quad (2.13)$$

The circuit for QPE is visible in Figure 2.3. The precision of the QPE algorithm depends on the number of qubits in the counting register. The more they are, the better the precision [10].

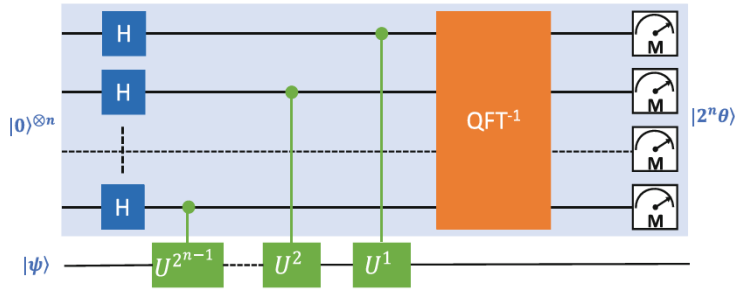


Figure 2.3: A circuit showing a general implementation of QPE with n qubits [7].

2.6 Shor's Algorithm

Shor's Algorithm is a quantum integer factorization algorithm developed in 1994 [1] by Peter Shor and has the potential to break encryption methods such as RSA that rely on generating a large integer number, N , by utilizing two large prime numbers, P and Q , meaning $N = P \times Q$ [11]. The prime integer factorization of the integer N can be done through a method known as period finding of the integer r through the function $f_{a,N}(x) = a^x \bmod N$ where a is an integer that is relatively prime to N [11]. Finding the period

r through modular exponentiation is generally a computationally demanding task for classical computers that would take exponential time and does not have any specific advantage over directly searching for the factors P and Q . However, with the usage of the Quantum Fourier Transform (QFT) on quantum hardware this can be done in polynomial time instead, greatly increasing the efficiency [2]. After finding the period r , the remaining part of the integer factorization with Shor's algorithm is computed classically by utilizing gcd (greatest common divisor) with the periodicity r , and the coprime integers a and N which from the numbers P and Q are finally found [11]. This means that Shor's algorithm has a quantum section for the order finding problem (period finding) and a purely classical section that does the factoring by utilizing the period.

Before describing how the order finding problem is computed on a quantum computer it is important to understand the idea behind it. As explained, the method of finding the period r is through the function

$$f_{a,N}(x) = a^x \mod N$$

where a is an integer that is relatively prime to N and the modulo operator returns the remainder a^x divided by N [11]. Furthermore, these two integers are relatively prime or coprime if $gcd(a, N) = 1$ where $1 < a < N$ where the gcd is the greatest common divisor. Through modular exponentiation it is possible to find the repeating period r . Check the following example:

The number 15 is to be factorized and it is found that 7 and 15 are relatively prime through $gcd(7, 15) = 1$, meaning $a = 7$ and $N = 15$. We will now find the period r by utilizing the function $f_{a,N}(x) = a^x \mod N$ and we begin by inputting 0 as x :

$$\begin{aligned}
f_{7,15}(0) &\equiv 7^0 \pmod{15} = 1 \\
f_{7,15}(1) &\equiv 7^1 \pmod{15} = 7 \\
f_{7,15}(2) &\equiv 7^2 \pmod{15} = 4 \\
f_{7,15}(3) &\equiv 7^3 \pmod{15} = 13 \\
f_{7,15}(4) &\equiv 7^4 \pmod{15} = 1 \\
f_{7,15}(5) &\equiv 7^5 \pmod{15} = 7 \\
f_{7,15}(6) &\equiv 7^6 \pmod{15} = 4 \\
f_{7,15}(7) &\equiv 7^7 \pmod{15} = 13 \\
f_{7,15}(8) &\equiv 7^8 \pmod{15} = 1 \\
&\vdots
\end{aligned}$$

Notice that the numbers are repeating during the modular exponentiation where $f_{7,15}(0) = f_{7,15}(0 + 4) = f_{7,15}(4 + 4) = 1$ meaning that the period r in this case is 4 [11].

Shor's algorithm is able to find the period r by utilizing a quantum circuit involving Quantum Phase Estimation (QPE) which contains two registers. The registers are set up by first choosing $T = 2^t$ such that $N^2 \leq T \leq 2N^2$ so that the upper register has t qubits while the lower register has n qubits where $n = \log_2 N$ [12]. The qubits in the upper register are initialized to $|0\rangle$ while the lower register is initialized to $|1\rangle$ [13]. The process begins with applying Hadamard gates on each qubit in the upper register. In the lower register, the modular exponentiation function is applied on every number in the lower register and then finally the quantum Fourier transform is applied on the upper register from which it is possible to measure the value z . By utilizing the continued fraction algorithm on $\frac{T}{z}$ there is a possibility to find an integer multiple of the period r which then, in combination with Euclid's algorithm, will find r [12].

After an even period r has been retrieved it is possible to factorize the integer N using the following properties of periodicity:

$$\begin{aligned}
a^r &\equiv 1 \pmod{N} \\
(a^{r/2})^2 &= a^r \equiv 1 \pmod{N} \\
(a^{r/2})^2 - 1 &\equiv 0 \pmod{N} \\
(a^{r/2} - 1)(a^{r/2} + 1) &\equiv 0 \pmod{N}
\end{aligned}$$

This implies that if the period r is even then $(a^{r/2} - 1)(a^{r/2} + 1)$ is a multiple of N and therefore there must be a nontrivial factor in either $(a^{r/2} - 1)$ or $(a^{r/2} + 1)$ also shared by N [1]. By utilizing this theorem we get that the prime factors of N can be calculated through $P = \gcd(a^{r/2} - 1)$ or $Q = \gcd(a^{r/2} + 1)$. However, if r is not even, it is necessary to redo the algorithm with a different value on a . [11]

2.7 Classical Algorithms

2.7.1 Trial division

Trial division is a classical integer factorization algorithm that relies on division of an integer N with all prime numbers $p < N$ in order to find its factors and takes $O(\sqrt{N})$ time. It is highly efficient for small composites but can take a long time factoring as the size of the composite increases [14]. The pseudo code can be written in the following way:

```

1 function trialDivision(N):
2   primes ← All primes < sqrt(N)
3   factors ← []
4
5   for each divisor in primes
6     while N % divisor == 0
7       factors.add(divisor)
8       N ← N/divisor
9
10  if factors == []
11    factors.add[N]
12  return factors

```

2.7.2 Pollard's Rho Algorithm

Pollard's Rho algorithm was introduced by John Pollard in 1975 [15] and is specialized for factoring numbers with small prime factors with a time complexity of $O(N^{1/4})$ [16]. The idea behind this algorithm is that the number to be factorized, N , has a small factor p and if there was a way to retrieve 2 integers x_i and x_j with the property of $x_i \equiv x_j \pmod p$ but not $\pmod N$ then the $\gcd(x_j - x_i, N)$ would be a nontrivial factor to N . In order to find the two integers, we generate a pseudorandom sequence $\pmod n$ that repeats $\pmod p$ where for instance $p(x) = x^2 - 1$ (more recently set as $x^2 + 1$) and $x_1 = 1$ [15].

In order to perform this algorithm and factorize an integer N assuming it has a small factor, set $x_0 = 2$, $c = 1$ and let the function $p(x)$ be $p(x) = x^2 - c = x^2 - 1$. In the first step, compute x_1 through the $p(x)$ function $\pmod N$ by substituting x with x_0 meaning $x_1 = p(x_0) \pmod N$ and continue doing this for x_2 , x_3 etc. For every even step in the algorithm, x_{2s} , compute the $\gcd(x_{2s} - x_s, N)$ and if the result of this is a number greater than 1 then the algorithm is finished [15]. The algorithm's time complexity of $O(N^{1/4})$ is derived from that if p is a prime, then it takes approximately $O(\sqrt{p})$ iterations to find p where $O(p) = O(N^{1/2})$ [16] [17]

The pseudocode of the most simple case of the algorithm can be written as the following:

```

1 function pollardRhoAlg(N):
2     a ← 2
3     b ← 2
4     c ← 1
5     d ← 1
6
7     while d == N:
8         a ← f(a, c) % N
9         b ← f(f(b, c), c) % N
10        d ← gcd(abs(a - b), N)
11
12    if d == N:
13        return factoring failed
14
15    return d

```

```

16
17 function f(x, c):
18     return x**2 + c

```

2.8 Related work

Kumar & Mondal [18] describe in their study on Shor's algorithm the progress and advancements made within the field of quantum factorization. In response to the challenges of efficiently solving traditionally difficult problems in factorization and discrete logarithm, Peter Shor showed that with the use of a quantum computer, these problems could be solved in polynomial time [18]. Prior to Peter Shor's suggestion of Shor's algorithm, there had been no polynomial-time algorithms discovered for the purpose of solving number theory problems regarding factorization and discrete logarithms [19]. Due to the sheer difficulty behind these problems, cryptographic systems, such as RSA, were fundamentally built on these concepts. Shor explains that the most efficient classical integer factorization algorithm known at the time was the General Number Field Sieve (GNFS) from the perspective of both time complexity and in practice. The algorithm is efficient for large integers and runs in exponential time when factoring a number n [19]. However, Shor shows that through the computational capabilities of quantum computing it is possible to solve these types of number theory problems in polynomial time, with a minor probability for error [19]. The time complexity of Shor's algorithm is in fact $O((\log n)^2(\log \log n)(\log \log \log n))$ on the quantum computer regarding the order finding problem and the amount of classical post-processing used for finding the factors of n being polynomial in $\log n$ [19]. This resulted in security concerns for cryptographic systems as they could be threatened by quantum computers which fueled further developments and advancements in quantum algorithms [18] [19].

Mounica et al., [10] describes the theory behind Shor's algorithm and implementations of the algorithm. In the theoretical part, a variety of quantum algorithms are explained such as QFT, IQFT and QPE but also the classical processing after the quantum part has been executed. Regarding their implementation of the algorithm, a quantum circuit with 5 qubits was created to factor the number 15. This was done with the help of IBM Qiskit. The coprime value was randomly generated and the most probable period value for r was 4, which in turn finds the correct factors for 15, namely 3 and 5. The factoring of

15 was also tried on the quantum computer `ibmq_santiago`, a 5-qubit system that has been retired since June 17, 2022. IBM's retired systems were once crucial for experiments with quantum algorithms and laid the groundwork for the modern systems [21]. The factorization of 15 showed that the experimental and theoretical results mostly matched each other. However, it is important to mention that the co-prime number to 15 for the `ibmq_santiago` implementation was 13. In addition, when the number 21 was factored, the theoretical and experimental results did not agree with one another. In contrast, the IBM Qiskit version used by Mounica et al. [10] also had a built-in function for simulating Shor's algorithm whose results for 15, 21 and 35 agreed with theoretical expectations. It should be noted that this built-in function of Shor's algorithm has been removed and is not available in newer versions of IBM Qiskit [22]. Furthermore, as of 2020, $N = 21$ is the highest number that has been factored on real quantum hardware [7].

DeCusatis & Mcgettrick [23] provides in their article an implementation of Shor's algorithm in Qiskit which was executed on both a simulator and on real quantum hardware. Similarly to other studies within the field of quantum integer factorization, only small numbers such as 15 and 21 were feasible to factorize due to the necessity of more qubits for larger integers. The results from the experiment involve taking the differences between simulation and real hardware, the IBMQ-16-Melbourne system (15-qubit system, retired since august 9, 2021) [21], into consideration and integrates noise as a part of the discussion. It must be noted that the noise of the quantum computer makes it harder to evaluate the period.

Beauregard [24] discusses how to reduce the number of qubits necessary to perform the order finding part of Shor's algorithm. It is described how to construct a variety of gates, such as the modular adder, the controlled multiplier, and the controlled swap, and how these gates are a part of his circuit for Shor's algorithm. This new circuit needs $2n + 3$ qubits to function, and Beauregard believes that it might be possible to reduce that number to less than $2n + O(1)$ qubits if another strategy is applied. Beauregard's implementation utilizes fewer qubits than many previously known variants and has a depth of $O(n^3)$. To factor number 15, only 11 qubits are needed with this circuit. This research also mentions that there is a good chance that the quantum part of Shor's algorithm can find the order r for a specific coprime modulo N . However, this part might need to run with different coprime values. Furthermore, it can take multiple attempts for a number to be properly factored. An inter-

esting point that Beauregard brings up is that it is unclear whether the number of qubits or the circuit depth is more important to take into consideration.

Hamdi et al. [25] conducted a comparative study between Shor's algorithm and the classical integer factorization algorithm known as GNFS. In order to perform GNFS in this study, a variety of software packages and components were used due to the implementation of the algorithm being immensely complex. These tools are able to perform certain steps of the classical algorithm and were all executed in a Python environment on the command prompt. During this experiment, Hamdi et al. chose a 100-digit integer to be factored by GNFS and reported that the total runtime was approximately 1 hour and 50 minutes. Furthermore, Hamdi et al. examined the temperatures of the CPU cores which after 24 minutes into the execution of the algorithm were above 80°C and identified that this could potentially damage the system over time. Regarding the comparison between GNFS and Shor's algorithm, Hamdi et al. initially explains that the time complexity of GNFS is $O(\exp[(\log n)^{1/3}(\log \log n)^{2/3}])$, where n is the input integer to be factored. This time complexity is super-polynomial (or sub-exponential depending on definition) which is slower than Shor's algorithm, which has a polynomial time complexity as described previously. Additionally, Hamdi et al. utilizes a figure scaling GNFS and Shor's algorithm with different clock rates in order to approximate the runtime of factoring a 100-digit integer with Shor's algorithm. This is because it is not possible to run these tests on actual quantum hardware as it is still being researched. It is shown that with a clock speed of 1 GHz it would take less than a second to factor that number with Shor's algorithm which is significantly faster than GNFS which took 1 hour and 50 minutes on a 2.20 GHz classical computer. Hamdi et al. concludes that Shor's algorithm is in theory better than classical algorithms such as GNFS and if in the future a quantum computer that can run Shor's algorithm on any integer number is implemented, cryptographic systems such as RSA are very likely to be solved.

Chapter 3

Method

3.1 Circuits and algorithms

In our case, the circuits that were necessary to implement in order to proceed with the experiment was the order finding section of Shor's algorithm that is tasked with finding the period r . This circuit makes use of the Quantum Phase Estimation circuit and the controlled-U gate which in this case corresponds to the multiplication of $a \bmod N$. Figure 3.1 displays the quantum circuit behind the order finding problem in Shor's algorithm.

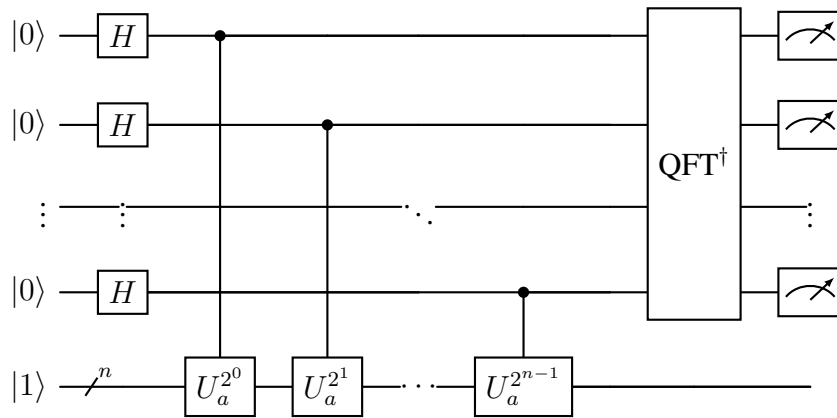


Figure 3.1: Quantum Phase Estimation circuit used in Shor's algorithm, applying controlled modular exponentiation and the inverse QFT.

This circuit implementation was implemented using two quantum registers, the upper register where all the qubits are initialized as $|0\rangle$ and the lower reg-

ister that is initialized as $|1\rangle$. In the upper register all qubits are put into superposition by applying the Hadamard gate operation, denoted as H . In this Quantum Phase Estimation circuit, the controlled U -gates correspond to the modular exponentiation function. Finally, after performing the inverse QFT in the upper register, the continued fraction algorithm is applied on the measured value in order to extract the period r .

Qiskit was used in order to implement the quantum circuits necessary to perform Shor's algorithm. The Qiskit SDK is an open source Python library suited for working with quantum computers and contains tools for implementing quantum circuits. The library has options for either running the quantum circuits on IBM quantum hardware through paid cloud technology or on your own machine by utilizing a quantum simulator [26]. Furthermore, the implementation of Shor's algorithm used in this experiment was obtained from the IBM Quantum Learning documentation which had an implementation for factoring $N = 15$ [27]. The code was modified in order to suit the purposes of this study.

It should be noted that the reason behind only factoring $N = 15$ is due to the limited amount of integers that an implementation of Shor's algorithm can factor in practice. Furthermore, the resource demands for running a quantum simulator increase significantly with the number of qubits used in the circuit. The circuit for $N = 15$ only requires 12 qubits and is often used to demonstrate and analyze the algorithm, as can be seen in the previously mentioned IBM Quantum Learning documentation. It was therefore considered a feasible value to conduct the research around.

The controlled multiplication circuit, being the controlled- U gates in Figure 3.1, is in our case hardcoded for $a \bmod 15$. This means that for the coprimes a that were considered for this experiment, the circuit has a predefined way of performing the SWAP operations depending on the coprime value that was chosen.

The classical integer factorization algorithms that were implemented were Pollard's Rho Algorithm and Trial division which were then compared to the Shor's algorithm. The pseudocode behind these two functions along with the theory can be found in *Section 2.7.1 & 2.7.2*. These algorithms do not make use of the Qiskit SDK as they are purely classical which do not require any quantum operations. The function for the Trial division algorithm is declared

as *trialDivision*(N , *runs*) where N is the integer to be factorized and *runs* corresponds to the number of times the algorithm should be executed. Pollard's Rho Algorithm is defined in a similar manner, being *pollardRho*(N , *runs*) where the parameters are the same as for the Trial division algorithm.

3.2 Experimental setup/Implementation

The computer used for the experiment had the following specifications:

- *CPU: 12th Gen Intel(R) Core(TM) i7-12700H*
- *GPU: NVIDIA GeForce RTX 3060 Laptop GPU*
- *RAM: 16 GB*
- *OS: Windows 11 Home*

The experiment was conducted in a Python environment consisting of an .ipynb file in Jupyter Notebook. The following versions were used for Python and the libraries:

- *Python 3.11.9*
- *Qiskit 1.4.0*
- *Pandas 2.2.2*
- *Matplotlib 3.9.0*
- *Seaborn 0.13.2*

The notebook format allowed for the code to be organized in a modular manner where the functions regarding the circuits, algorithms and metric evaluation could be contained within their own respective code cells. Furthermore, this approach simplified the visualization of the circuits and the display of the various metrics of interest. Said metrics consisted of the average runtime of each algorithm and in the case of Shor's algorithm, the number of attempts to factor 15 and the success ratio (number of successful attempts / the total number of attempts) as well. To obtain this information, several functions were created to run the different factorization algorithms which were executed 10 000 times each to get representative data. For every algorithm, in the beginning of a run,

a timer was initiated to keep track of the time the run took. When the current algorithm attempt completed, the timer stopped. All these runtimes were stored in an array. In the case of Shor's algorithm, the metrics were collected for each coprime value a to analyze whether they caused any differences in the results. As mentioned earlier, due to the possibility of the algorithm failing to find factors when factorizing a number N , the number of attempts needed for each run to find a factor were also counted and stored in an array. In addition, we kept track of each run's number of failed attempts and the total attempts which Shor's algorithm made for each respective coprime a . However, for the classical algorithms it is not necessary to count the number of attempts as they will always succeed for small integers like 15 without having to rerun the algorithm. Therefore, the classical algorithms only tracked and returned the runtime of each run and a list of factors (only used for debugging purposes).

3.3 Evaluation of the metrics

As mentioned in the previous section, the metrics were stored in various arrays and variables that were returned from each factorization algorithm's respective function. More specifically, Shor's algorithm returns the total number of attempts made during the 10 000 runs, a list containing the runtime of each run, a list with all the failed attempts per run and a list containing all the attempts that were made per run whereas both of the classical algorithms only returned a list of runtimes and factors. All these arrays and variables, apart from the factors which served solely a debugging purpose, were used to perform calculations and to setup the plotting material.

Shor's algorithm gathered the metrics for the various coprime values a . After each coprime a completed Shor's algorithm, the average time per run was calculated before being appended to an array of average runtimes. Afterwards, a for loop was applied to get the data of each run saved in a dictionary. This dictionary saved information relating to the algorithm name, the number of the run and its runtime, the current coprime value and the number of successful, failed and total attempts for each respective run. The next step consisted of calculating the success ratio of Shor's algorithm and then appending it to a list. After all coprime values had been used in Shor's algorithm and their data had been put in the dictionary, a similar process was initiated for the classical algorithms. For both classical algorithms, after they had finished executing, the average time per run was calculated. Then the data of each run was put in their respective dictionary (there were different dictionaries for Pollard's Rho

and Trial division). Unlike Shor's algorithm however the dictionaries of the classical algorithms only kept track of the algorithm name, the number of the run and the runtimes.

When all algorithms had been run and the setup had finished, the dictionaries were placed in two Pandas data frames, one for Shor's algorithm and one for the classical algorithms. The data frames were then used in functions to generate relevant plots from the data gathered during the experiment. In addition, the variables/lists that contained the values for the success ratio and average runtime of Shor's algorithm as well as the average runtimes of the classical algorithms were printed so that their results could be put into tables.

Many of the results were visualized in various charts and diagrams that were made using the Python libraries Pandas, Matplotlib, and Seaborn. Bar plots were used to illustrate the average runtime that was required for each coprime a when running Shor's algorithm. This was done to see if any a value gave a different comparison to the classical algorithms. The error was also included in the plot to get an understanding of the variance between different runtimes for each coprime. In addition, the average runtime for all runs of Shor's algorithm was calculated, regardless of the value of a , to get a picture of how the algorithm compares to the classical algorithms in general. This average was plotted in a bar plot together with the average runtimes for the Pollard's Rho algorithm and Trial division (using a logarithmic scale). Furthermore, another bar plot was generated, visualising both the number of successful and failed attempts for each coprime a to get a display of the success ratio. Lastly a scatterplot was made for Shor's algorithm to see whether there is a link between the average runtime for each coprime and their corresponding success ratio.

Chapter 4

Results

This section presents the results of the experiment in accordance with the aims and purposes of the study. The research question aimed to compare Shor's algorithm with classical integer factorization algorithms from the perspective of accuracy and runtime. Table 4.1 provides an overview of the average runtime of the quantum (Shor) and classical algorithms (Pollard's Rho & Trial division) across all 10 000 iterations when factoring $N = 15$. Shor's algorithm in particular is a probabilistic factorization algorithm that utilizes a number a that is coprime to N in the order finding problem. Therefore, the algorithm was executed using multiple integers that are coprime to $N = 15$, which consisted of $a = [2, 4, 7, 8, 11, 13]$. It is apparent from this table that the Pollard's Rho algorithm and Trial division are significantly faster than Shor's algorithm when factoring the number 15, regardless of the value for the coprime a .

Algorithm	Avg. Runtime (s)
Shor's $a = 2$	0.672
Shor's $a = 4$	0.806
Shor's $a = 7$	0.632
Shor's $a = 8$	0.583
Shor's $a = 11$	0.990
Shor's $a = 13$	0.640
Trial Division	2.94e-7
Pollard's Rho	4.56e-7

Table 4.1: The average runtime of each algorithm. Shor's Algorithm was executed on different values for coprime a and each algorithm was executed 10 000 times

Additionally, Figure 4.1 illustrates the differences in the average runtime between Shor’s algorithm and the classical algorithms in the form of a log scale. Note that in this figure, the mean runtime in Shor’s algorithm for all coprime values a is used to display the average runtime. Evidently, the runtime of Shor’s algorithm is longer than both of the classical algorithms by a significant margin.

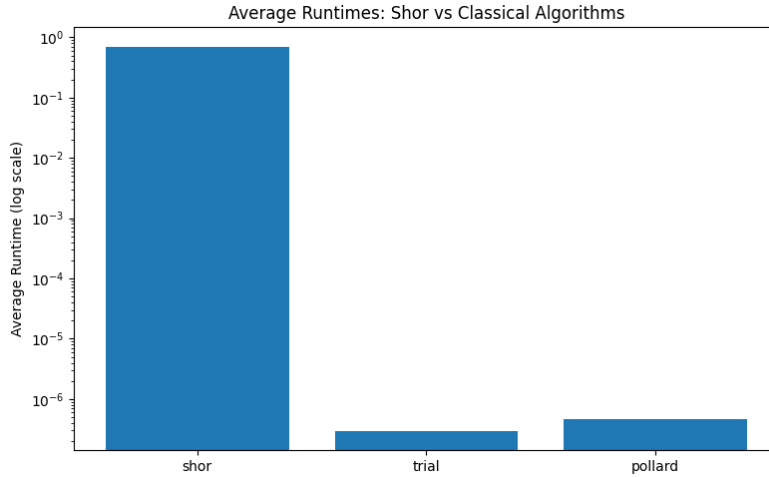


Figure 4.1: Comparison of average runtimes between Shor’s algorithm and classical algorithms (log scale).

If we now turn to the variety in factorization performance among different coprime values in Shor’s algorithm, Figure 4.2 provides the total number of failed and successful attempts in a stacked bar chart. What stands out in this chart is that the number of failed attempts were at their highest for $a = [4, 11]$ and at their lowest for $a = [2, 7, 8, 13]$ which had relatively similar performances. In Figure 4.3, the scatter plot shows the correlation between the number of attempts and the runtime for different coprimes values a . The points that are near the top left have the highest success ratio and lowest runtime while the bottom right corner displays the opposite effect. This implies similar results as Figure 4.2, meaning that $a = [2, 7, 8, 13]$ had the best performance while $a = [4, 11]$ had the worst performance. Interestingly, $a = 8$ had the lowest runtime even though the success ratio was slightly lower than $a = [7, 13]$.

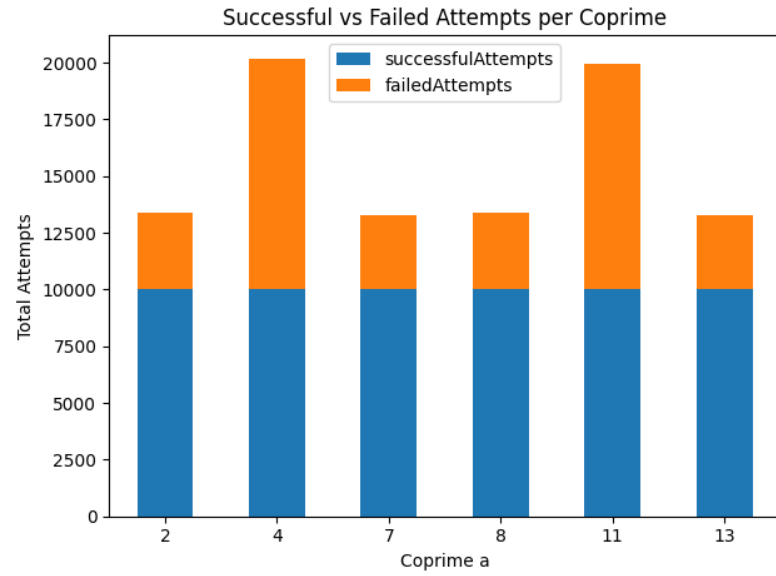


Figure 4.2: Total attempts on Shor's algorithm for each coprime a in the form of a stacked plot that displays the number of successful and failed attempts across all iterations of the experiment.

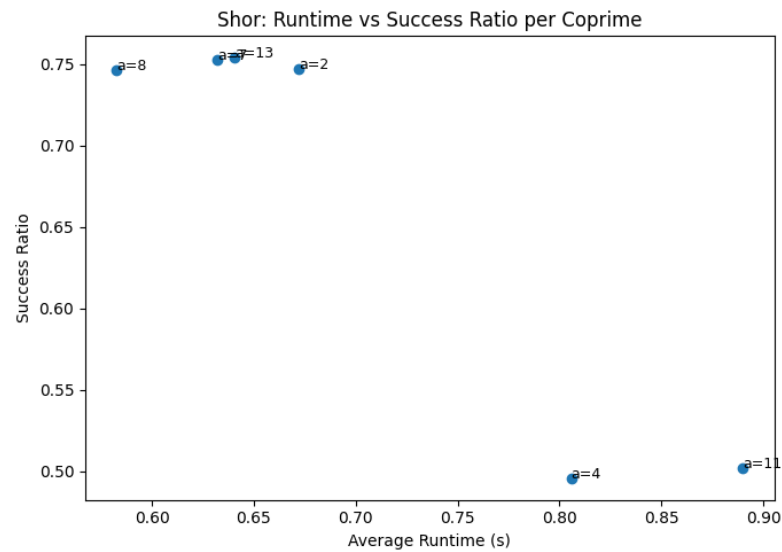


Figure 4.3: Scatter plot displaying the correlation between the runtime and number of attempts for each coprime a in Shor's algorithm.

In order to highlight the differences in success rate between different coprime numbers used in Shor's algorithm, Table 4.2 provides an overview of the success ratio. In particular, $a = [4, 11]$ have the lowest success ratio while the remaining coprime values on a have a relatively similar rate hovering around 0.750.

Algorithm	Success Ratio
Shor's $a = 2$	0.747
Shor's $a = 4$	0.495
Shor's $a = 7$	0.752
Shor's $a = 8$	0.746
Shor's $a = 11$	0.501
Shor's $a = 13$	0.754

Table 4.2: The success ratio of Shor's algorithm when factoring $N = 15$ for different coprime values.

However, it is important to consider the standard deviation of the runtime of Shor's algorithm for different coprimes which can be viewed in Figure 4.4. It is apparent even here that coprimes $a = [4, 11]$ had the highest amount of variation around their mean value.

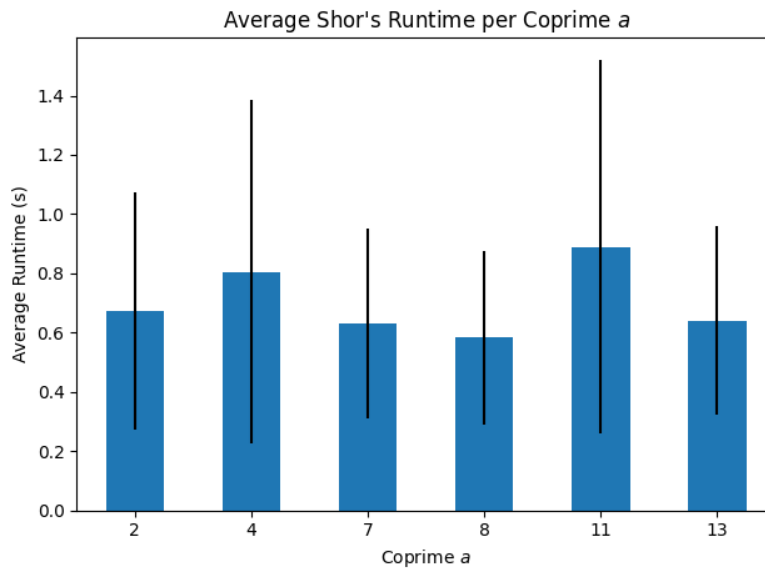


Figure 4.4: Bar chart with standard deviation regarding the runtime of Shor's algorithm for different coprimes a .

Chapter 5

Discussion

It becomes evident from the results section, more specifically Table 4.1 and Figure 4.1, that the runtime of Shor's algorithm is much slower compared to the classical algorithms regardless of coprime values. One explanation for this is the probabilistic nature of Shor's algorithm, which means that it might be necessary for the algorithm to run multiple times before successfully factoring the number 15. This is further supported by Figure 4.2, which displays the number of failed attempts for each coprime a . Therefore, it naturally takes more time to find the correct factors. Another reason for the longer runtimes of Shor's algorithm may be due to the experiment being conducted using a Qiskit simulator and not actual quantum hardware. Thus, running this simulator may have caused an overhead effect which in turn caused Shor's algorithm take more time. However, as noted in the study conducted by Hamdi et al. [25], Shor's algorithm would in theory be significantly faster than the classical approaches when factoring very large integers on a fully fledged quantum computer. In the study by Hamdi et al., the runtime of a classical algorithm, GNFS, for factoring a 100 digit number was compared with the theoretical runtime of Shor's algorithm which was approximated with a graph. It takes approximately less than a second for Shor's algorithm to factor the 100 digit integer with a clock speed of 1 GHz while the runtime of GNFS was an hour and 50 minutes [25]. In our case, as the number to be factored was as small as $N = 15$ and the setup was a simulator, Shor's algorithm was instead performing worse than the classical algorithms.

Furthermore, the results show that Shor's algorithm had worse accuracy compared to classical algorithms. This can also be attributed to Shor's algorithm being probabilistic since, as mentioned earlier, the algorithm might need to run

multiple times before successfully factoring N . The more attempts were made, the lower the final accuracy ratio ended up being, as can be seen in figures 4.2. It should be noted that Pollard's Rho algorithm is probabilistic as well which does imply that it can fail depending on the circumstances. However, due to the parameter values chosen in our implementation, it will not happen in this case of factoring $N = 15$. Despite this, in some versions of the algorithm these values are randomized. If this study were to consider multiple integers to be factored, it is almost certain that the parameter values for Pollard's Rho algorithm would have to be randomized until a successful factorization would occur.

Additionally, it is worth mentioning the differences between the various coprime values for Shor's algorithm. Most values for a had similar success ratio, around 0.750 except for $a = [4, 11]$ which both had an accuracy around 0.50 as can be seen in Table 4.2. Furthermore, regarding the execution runtime, there is a greater difference among the coprime values. However, once again $a = [4, 11]$ are the most significant outliers, having a much longer runtime than the other coprimes (see Figure 4.2 & Figure 4.3). This can partly be explained by the higher variance exhibited by their respective runtimes which is visible in Figure 4.4. The reason behind this is likely that Shor's algorithm is failing more often to find a valid period when using these specific coprime values in the order finding problem. It should be noted that $a = [4, 11]$ both have a period of $r = 2$ whereas the remaining coprimes all have a period of $r = 4$. This suggests that it is more difficult to factor 15 with a coprime value that has period $r = 2$ than a coprime integer with a period of $r = 4$. There is always a risk that Shor's algorithm will not return the correct period r but it can still be worth trying this value out since it might find a correct factor of 15 after all. This is the case for many of the values $a = [2, 7, 8, 13]$ which have the period $r = 4$. For instance, when $a = 7$, the quantum part of Shor's algorithm might give $r = 2$, which is not the period value for 7. Despite this, when using $r = 2$, $a = 7$, and $N = 15$, we obtain the values 6 and 8 from $a^{r/2} - 1$ and $a^{r/2} + 1$ respectively. Thus, since $\gcd(6, 15) = 3$, we can factor 15. The same phenomenon cannot be observed with $a = [4, 11]$, where any value that is not the period is uneven and will therefore lead to a failed attempt. For this reason, the criteria for a successful factorization are stricter when working with $a = [4, 11]$. A similar observation was made by Mounica et al. [10] who noted that the experiments conducted with their own 5-qubit implementation of Shor's algorithm showed that $r = 4$ was the most probable period value. Unlike our implementation, which sought to see how coprime

values to 15 affected runtime and accuracy, Mounica et al. made use of a randomized coprime value for running their implementation of Shor's algorithm. Therefore, from our own observations regarding the success ratio for different coprimes and since $r = 4$ is the most probable period, it is likely that a randomly chosen coprime value with a period of $r = 4$ will lead to a successful factorization when trying to factor 15.

5.1 Societal impact and Industrial relevance

This research shows the current state of Shor's factorization algorithm compared to classical algorithms, such as Trial division and Pollard's Rho algorithm. It provides a detailed comparison regarding the differences between the two approaches that is easy to replicate while also drawing attention to the challenges that may arise when working with quantum computing. Our work shows that the classical methods remain faster for a small integer, $N = 15$, and demonstrates the current gap between quantum and classical integer factorization algorithms in practice. In industry, this implies that quantum factorization is currently unsuitable for practical factorization tasks. However, in the long term, if a fully capable Shor algorithm were implemented, it would have an immense impact on both industry and society. Widely used encryption systems, such as RSA, make use of massive integer values that take a long time to factor via classical approaches. A successful Shor's algorithm on the other hand could perform the factorization much faster which would then lead to these encryption systems becoming unsafe. This is also acknowledged by the creator of the algorithm, Peter Shor [19].

5.2 Ethical aspects and Sustainability

The study involves important ethical aspects behind quantum computing in regards to encryption and information security. Although the results demonstrate that Shor's algorithm is currently slower than the classical algorithms for $N = 15$, the theoretical capabilities when factoring larger integers point toward that quantum computers in the future will pose a threat towards cryptographic systems such as RSA. The ethical implications of this involve risks regarding privacy and data. However, even though Shor's algorithm theoretically renders encryption systems insecure, it does contribute towards the UN sustainability development goals. In our case, SDG 9: *Build resilient infrastructure, promote inclusive and sustainable industrialization and foster inno-*

vation [28]. Shor's algorithm will contribute towards pushing innovation in order for the current day systems and infrastructure to implement solutions to deal with the security threats caused by quantum computing.

5.3 Limitations and Future work

One limitation with this study was our use of a simulator rather than real quantum hardware. To get more representative results, it would be better to make use of a real quantum computer. Another limitation regarding the comparison of Shor's algorithm versus the classical algorithms, is the fact that quantum computing is still in its early stages. Due to Shor's algorithm only being able to reliably factor small numbers like 15, it is difficult to make a proper comparison between it and the classical counterparts. The classical algorithms have always been fast for the factorization of smaller values. The real challenge for them is to factor large numbers which Shor's algorithm in theory should be able to do much faster. Therefore, to obtain more meaningful results, it would be necessary to perform our experiment with larger numbers that are difficult for classical algorithms to factor. Future implementations of this study should take into consideration the points mentioned and make use of proper quantum hardware while also factoring larger integer values. Although these proposals would lead to more statistically significant results, further advancements within quantum computing and hardware are necessary before such a study can be conducted in practice.

Chapter 6

Conclusion

The purpose of this study was to investigate how Shor's algorithm compares to various classical algorithms such as Trial division and Pollard's Rho algorithm in terms of runtime and accuracy. By utilizing the Qiskit framework, it was possible to implement Shor's algorithm and perform this comparison with the classical counterparts when factoring $N = 15$. By examining the results, it is possible to conclude that Shor's algorithm is in practice performing worse than the classical algorithms within the scope of this study. However, previous research has shown that in theory, when factoring large integers, Shor's algorithm should be performing better. It appears that further advancements in quantum computing are necessary before Shor's algorithm will be able to factor larger integers, outperform the classical counterparts and pose a risk to cryptographic systems such as RSA.

Bibliography

- [1] M. Hayward. Quantum Computing and Shor's Algorithm. [Online]. 2005. Available:
<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=c4c3ad4aef68f3970d187fec0f13755471579018>
- [2] A. J. et al. "Quantum Algorithm Implementations for Beginners" in *ACM Transactions on Quantum Computing*. vol 3, no 4. 2022. pp. 1-92. doi: <https://doi.org/10.48550/arXiv.1804.03719>
- [3] E. Grumblin and M. Horowitz. "Progress in Computing" in *Quantum Computing: Progress and Prospects*. Washington, DC. United States of America. The National Academies Series Press. 2019. pp. 12-22. doi: <https://doi.org/10.17226/25196>
- [4] T. Thamaraimanalan, B. Singh, M. Mohankumar, and S. K. Korada. "Performance Analysis of Shor's Algorithm for Integer Factorization Using Quantum and Classical Approaches" in *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*. 2024: pp. 2591-2595. doi: <https://doi.org/10.1109/ICACCS60874.2024.10717174>
- [5] C. Hughes, J. Isaacson, A. Perry, R. F. Sun and J. Turner, *Quantum Computing for the Quantum Curious*, 1st edition. Cham, Switzerland: Springer Cham, 2021, pp. 7-11 and 53-55. Available: <https://link.springer.com/book/10.1007/978-3-030-61601-4>
- [6] R. Kharsa, A. Bouridane, and A. Amira, Eds., "Advances in Quantum Machine Learning and Deep Learning for Image Classification: A Survey" in *Neurocomputing*. vol. 560, 2023. doi: <https://doi.org/10.1016/j.neucom.2023.126843>
- [7] V. Kasirajan, *Fundamentals of Quantum Computing: Theory and Practice*, 1st Edition. Cham, Switzerland: Springer Cham, 2021. pp.

93-95, 181-183, 229, 346-348. Available:
<https://link-springer-com.focus.lib.kth.se/book/10.1007/978-3-030-63689-0>

- [8] R. K. Bera, *The Amazing World of Quantum Computing*, 1st Edition. Singapore: Springer Singapore, 2020, pp. 69, 144 and 218-221. Available: <https://link-springer-com.focus.lib.kth.se/book/10.1007/978-981-15-2471-4>
- [9] L. Ruiz-Perez and J. C. Garcia-Escartin, Eds., "Quantum arithmetic with the quantum Fourier transform" in *Quantum Inf Process*, vol. 6, no. 152, 2017. pp. 2-3. doi: <https://doi.org/10.1007/s11128-017-1603-1>
- [10] G. R. Mounica, G. Manimaran, L. B. Jerome and P. Bhattacharjee, "Implementation of 5-Qubit approach-based Shor's Algorithm in IBM Qiskit" in *2021 IEEE Pune Section International Conference (PuneCon)*. 2021 pp. 1-6. doi: <https://doi.org/10.1109/PuneCon52575.2021.9686492>
- [11] H. Y. Wong. "Shor's Algorithm" in *Introduction to Quantum Computing*. Springer. 2023. pp. 289-298. doi: https://doi.org/10.1007/978-3-031-36985-8_29
- [12] Qiskit Community, P. Anna. "Shor's Algorithm for Integer Factorization." Accessed: April 4th, 2025. [Online]. Github Repository. 2019. Available: https://github.com/qiskit-community/qiskit-community-tutorials/blob/master/algorithms/shor_algorithm.ipynb
- [13] D. Sun, N. Zhang, and F. Franchetti. "Optimization and Performance Analysis of Shor's Algorithm in Qiskit" in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. 2023. pp. 1-7. doi: <https://doi.org/10.1109/HPEC58863.2023.10363522>
- [14] P. L. Jensen. Integer Factorization. M. S. thesis, Department of Computer Science, University of Copenhagen, Denmark. 2005. [Online]. Available: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=89b8adb2f3cd6c809f98cc5f1be676974ca48b22>

- [15] J. Wyss-Gallifent. (2023). Pollard's Rho Method. [Online]. Available: <https://math.umd.edu/~immortal/MATH406/lecturenotes/pollardrho.pdf>
- [16] P. L. Montgomery. "A Survey of Modern Integer Factorization Algorithms" in *CWI Quarterly*. vol 7, no 4. 1994. pp. 337-365. [Online]. Available: <https://core.ac.uk/download/pdf/301669364.pdf>
- [17] M. Bläser and C, Saha. (2012). PDF lecture. [Online]. Available: <https://www.csa.iisc.ac.in/~chandan/courses/CNT/notes/lec18.pdf>
- [18] M. Kumar and B. Mondal. "Study on Implementation of Shor's Factorization Algorithm on Quantum Computer" in *SN Computer Science*. 2024. vol 5, no 413. doi: <https://doi.org/10.1007/s42979-024-02771-y>
- [19] P. W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer" in *SIAM Review*. 1999. vol 41, no 2. pp 303-332. doi: <https://doi.org/10.1137/S0036144598347011>
- [20] Jack D. Hidary. "Qubits, Operators and Measurement" in *Quantum Computing: An Applied Approach*, 2nd edition. Cham, Switzerland: Springer Cham, 2021, ch 3. pp. 26. Available: https://link-springer-com.focus.lib.kth.se/chapter/10.1007/978-3-030-83274-2_3
- [21] M. AbuGhanem. "IBM Quantum Computers: Evolution, Performance, and Future Directions". 2024. arXiv: 2410.00916.
- [22] IBM Quantum Documentation. Algorithms migration guide. Accessed: May 28th, 2025. [Online]. Available: <https://docs.quantum.ibm.com/migration-guides/qiskit-algorithms-module>
- [23] C. DeCusatis and E. Mcgettrick, "Near term implementation of Shor's Algorithm using Qiskit," in *2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC)*. 2021. NV, USA. pp. 1564-1568, doi: <https://doi.org/10.1109/CCWC51732.2021.9376169>.

- [24] S. Beauregard. "Circuit for Shor's algorithm using $2n+3$ qubits". 2003. arXiv: 0205095v3.
- [25] S. M. Hamdi, S. T. Zuhori, F. Mahmud and B. Pal, "A Compare between Shor's quantum factoring algorithm and General Number Field Sieve" in *2014 International Conference on Electrical Engineering and Information & Communication Technology*, pp. 1-6, doi: <https://doi.org/10.1109/ICEEICT.2014.6919115>
- [26] IBM Quantum Documentation. "Introduction to Qiskit." Accessed: March 30th, 2025. [Online]. Available: <https://docs.quantum.ibm.com/guides>
- [27] IBM Quantum Documentation. "Phase Estimation and Factoring." Accessed: March 6th, 2025. [Online]. Available: <https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/phase-estimation-and-factoring#implementation-in-qiskit>
- [28] Global Goals. "Goal 9: Industry, innovation and infrastructure." Accessed: June 29th, 2025. [Online]. Available: <https://globalgoals.org/goals/9-industry-innovation-and-infrastructure/>

Appendix A

Source Code

```
1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit.circuit.library import QFT
3 from qiskit.primitives import Sampler
4 from qiskit.visualization import plot_histogram
5 import math
6
7 # Specialized circuit for factoring the number 15 from Qiskit learning docs
8 # https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/
   phase-estimation-and-factoring#implementation-in-qiskit (2025-03-06)
9 def c_amod15(a):
10     """
11     Controlled multiplication by a mod 15.
12     This is hard-coded for simplicity.
13     """
14     if a not in [2, 4, 7, 8, 11, 13]:
15         raise ValueError("'a' must not have common factors with 15")
16     U = QuantumCircuit(4)
17     if a in [2, 13]:
18         U.swap(2, 3)
19         U.swap(1, 2)
20         U.swap(0, 1)
21     if a in [7, 8]:
22         U.swap(0, 1)
23         U.swap(1, 2)
24         U.swap(2, 3)
25     if a in [4, 11]:
26         U.swap(1, 3)
27         U.swap(0, 2)
28     if a in [7, 11, 13]:
29         for q in range(4):
30             U.x(q)
31     U = U.to_gate()
32     U.name = f"{a} mod 15"
33     c_U = U.control()
34     return c_U
35
36 # Phase estimation procedure from the Qiskit learning docs
37 # https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/
   phase-estimation-and-factoring#implementation-in-qiskit (2025-03-06)
38 def phase_estimation(
```

```

39         controlled_operation: QuantumCircuit,
40         psi_prep: QuantumCircuit,
41         precision: int
42     ):
43         """
44         Carry out phase estimation on a simulator.
45         Args:
46             controlled_operation: The operation to perform phase estimation on,
47                                   controlled by one qubit.
48             psi_prep: Circuit to prepare  $|\psi\rangle$ 
49             precision: Number of counting qubits to use
50         Returns:
51             float: Best guess for phase of  $U|\psi\rangle$ 
52         """
53         control_register = QuantumRegister(precision)
54         output_register = ClassicalRegister(precision)
55
56         target_register = QuantumRegister(psi_prep.num_qubits)
57         qc = QuantumCircuit(control_register, target_register, output_register)
58
59         # Prepare  $|\psi\rangle$ 
60         qc.compose(psi_prep,
61                   qubits=target_register,
62                   inplace=True)
63
64         # Do phase estimation
65         for index, qubit in enumerate(control_register):
66             qc.h(qubit)
67             for _ in range(2**index):
68                 qc.compose(
69                     controlled_operation,
70                     qubits=[qubit] + list(target_register),
71                     inplace=True,
72                 )
73
74             qc.compose(
75                 QFT(precision, inverse=True),
76                 qubits=control_register,
77                 inplace=True
78             )
79
80         qc.measure(control_register, output_register)
81
82         measurement = Sampler().run(qc, shots=1).result().quasi_dists[0].popitem()
83         return measurement / 2**precision
84
85     # Prepareing the lower register of the QPE circuit
86     # https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/phase-estimation-and-factoring#implementation-in-qiskit (2025-03-06)
87     psi_prep = QuantumCircuit(4)
88     psi_prep.x(0)
89     psi_prep.draw("mpl")
90
91     # Importing libraries for evaluating the results and such
92     from fractions import Fraction
93     from math import gcd
94     import time
95     from datetime import datetime

```

```

96 import statistics
97 import pandas as pd
98 import matplotlib.pyplot as plt
99 import seaborn as sns
100
101 # Shor's algorithm, modified to measure certain metrics and run multiple
    iterations
102 # https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/
    phase-estimation-and-factoring#implementation-in-qiskit (2025-03-06)
103 def runShorsAlgorithm(N, a, runs):
104     attemptsList = []
105     totalAttempts = 0
106     runtimes = []
107     failedAttempts = []
108
109
110     for i in range(0, runs):
111         start = time.perf_counter() # Measure runtime
112         FACTOR_FOUND = False
113         ATTEMPT = 0
114         while not FACTOR_FOUND:
115             ATTEMPT += 1
116
117             phase = phase_estimation(
118                 c_amod15(a),
119                 psi_prep,
120                 precision=8
121             )
122             frac = Fraction(phase).limit_denominator(N)
123             r = frac.denominator
124             #print(f"Period from phase estimation: r = {r}")
125             if phase != 0:
126                 # Previously only did the following which only got one of the
    factors:
127                 # Guess for a factor is  $\gcd(x^{r/2} - 1, 15)$ 
128                 #guess =  $\gcd(a ** (r // 2) - 1, N)$ 
129                 guesses = [ $\gcd(a ** (r // 2) - 1, N)$ ,  $\gcd(a ** (r // 2) + 1,$ 
N)]
130                 #print(f"Factors in guesses array: {guesses[0]} & {guesses
    [1]}") # From github
131                 for guess in guesses:
132                     if guess not in [1, N] and (N % guess) == 0:
133                         FACTOR_FOUND = True
134                 attemptsList.append(ATTEMPT) # Append the number of attempts of the
    current run
135                 totalAttempts += ATTEMPT # Add number of attempts for total attempts
136                 end = time.perf_counter() # Measure runtime
137                 runtimes.append(end - start) # Append runtime of the current run to
    array
138
139     # To get the failed attempts for every run
140     for attempts in attemptsList:
141         failedAttempts.append(attempts-1)
142
143
144     return totalAttempts, runtimes, attemptsList, failedAttempts
145
146 # Will see if we even use this
147 def getAvgRuntime(runtime, runs):

```

```

148     avgRuntime = runtime / runs
149     return avgRuntime
150
151 def trialDivision(N, runs):
152     # Since we are only factoring N = 15 this won't change much but
153     # technically it should be a function
154     # that calculates all primes < math.ceil(sqrt(N))
155     primes = [2, 3]
156     runtimes = []
157
158     for i in range(0, runs):
159         start = time.perf_counter() # Measure runtime
160         factors = []
161         for divisor in primes:
162             while ((N % divisor) == 0):
163                 factors.append(divisor)
164                 N = N / divisor
165
166         # In the case where the remaining N after trial division is a prime
167         # factor by itself, add it as a factor
168         if (N != 1):
169             factors.append(N)
170
171         # If N is prime from the beginning
172         if (factors == []):
173             factors.append(N)
174
175         end = time.perf_counter() # Measure runtime
176         runtimes.append(end - start) # Append runtime of current run to array
177
178     # factors only used for testing, not necessary to return or keep track of
179     # when running this experiment
180     return factors, runtimes
181
182 # This function corresponds to the f(x) function in Pollard's rho algorithm.
183 # Chose c = 1
184 def f(x, c):
185     result = x ** 2 + c
186     return result
187
188 # Pollard's Rho algorithm
189 # Also pollard can technically fail for certain integers depending on the
190 # value of a, b and c but if you randomize them on every failure
191 # you can get the correct factors. It does not fail with these values on N =
192 # 15
193 def pollardRho(N, runs):
194     runtimes = []
195     for i in range(0, runs):
196         start = time.perf_counter() # Measure runtime
197         factors = []
198         a = 2
199         b = 2
200         c = 1
201         d = 1
202
203         if (N == 1):
204             return N

```

```

201     # Pollard rho cannot factorize even numbers, if the number is even we
    know it has a factor of 2
202     while (N % 2 == 0):
203         factors.append(2)
204         N = N // 2
205
206
207     while (d == 1):
208         a = f(a, c) % N
209         b = f(f(b, c) % N, c) % N
210         d = gcd(abs(a - b), N)
211
212     factors.append(d)
213     end = time.perf_counter() # Measure runtime
214     runtimes.append(end - start)
215
216     # factors only used for testing, not necessary to return or keep track of
    when running this experiment
217     return factors, runtimes
218
219 # Gather metrics from the experiment
220 coprimes = [2, 4, 7, 8, 11, 13]
221 N = 15
222 runs = 10000
223 avgRuntimes = []
224 resultsShor = []
225 resultsClassical = []
226 attemptsRatio = []
227 failedTotal = 0
228
229 print(f"Gathering data from Shor's Algorithm on N = {N}")
230
231 for a in coprimes:
232     attempts, shorRuntimes, attemptsList, failedAttempts = runShorsAlgorithm(
        N, a, runs)
233     avgRuntime = statistics.mean(shorRuntimes) # This saves the mean of all
    runtimes for coprime a
234     avgRuntimes.append(avgRuntime) # We save the avg runtime per coprime a in
    this array
235     for i, rt in enumerate(shorRuntimes): # We append each singular run of
    shor's and save metrics in resultsShor
236         resultsShor.append({
237             'algorithm': 'shor',
238             'coprime': a,
239             'run': i + 1,
240             'runtime': rt,
241             'attempts': attemptsList[i],
242             'failedAttempts': failedAttempts[i],
243             'successfulAttempts': attemptsList[i] - failedAttempts[i]
244         })
245     for i in failedAttempts:
246         failedTotal += i
247     attemptsRatio.append((attempts - failedTotal) / attempts) # Calculating the
    success ratio for each coprime a
248     failedTotal = 0 # Resetting failedTotal for the next coprime a
249     print(f"Number of attempts for coprime {a}: {attempts}")
250     print(f"All runtimes for coprime {a}: {shorRuntimes}")
251
252     print(f"Average runtimes for each a in Shor's algorithm: {avgRuntimes}")

```



```

253
254 print(f"Gathering data from Trial Division on N = {N}")
255 trialFactors, trialdivRuntimes = trialDivision(N, runs)
256 trialdivAvg = statistics.mean(trialdivRuntimes) # This saves the mean of all
      runtimes
257 for i, rt in enumerate(trialdivRuntimes):
258     resultsClassical.append({
259         'algorithm': 'trial',
260         'run': i + 1,
261         'runtime': rt
262     })
263 print(f"All runtimes for Trial division: {trialdivRuntimes}")
264 print(f"Average runtime for Trial division: {trialdivAvg}")
265
266 print(f"Gathering data from Pollard's Rho Algorithm on N = {N}")
267 pollardFactors, pollardRuntimes = pollardRho(N, runs)
268 pollardAvg = statistics.mean(pollardRuntimes) # This saves the mean of all
      runtimes
269 for i, rt in enumerate(pollardRuntimes):
270     resultsClassical.append({
271         'algorithm': 'pollard',
272         'run': i + 1,
273         'runtime': rt
274     })
275 print(f"All runtimes for Pollard's Rho Algorithm: {pollardRuntimes}")
276 print(f"Average runtime for Pollard's Rho Algorithm: {pollardAvg}")
277
278 # Convert to DataFrame for plotting
279 dfShor = pd.DataFrame(resultsShor)
280 dfClassical = pd.DataFrame(resultsClassical)
281
282 print(attemptsRatio)
283 print(avgRuntimes)
284 print(trialdivAvg)
285 print(pollardAvg)

```